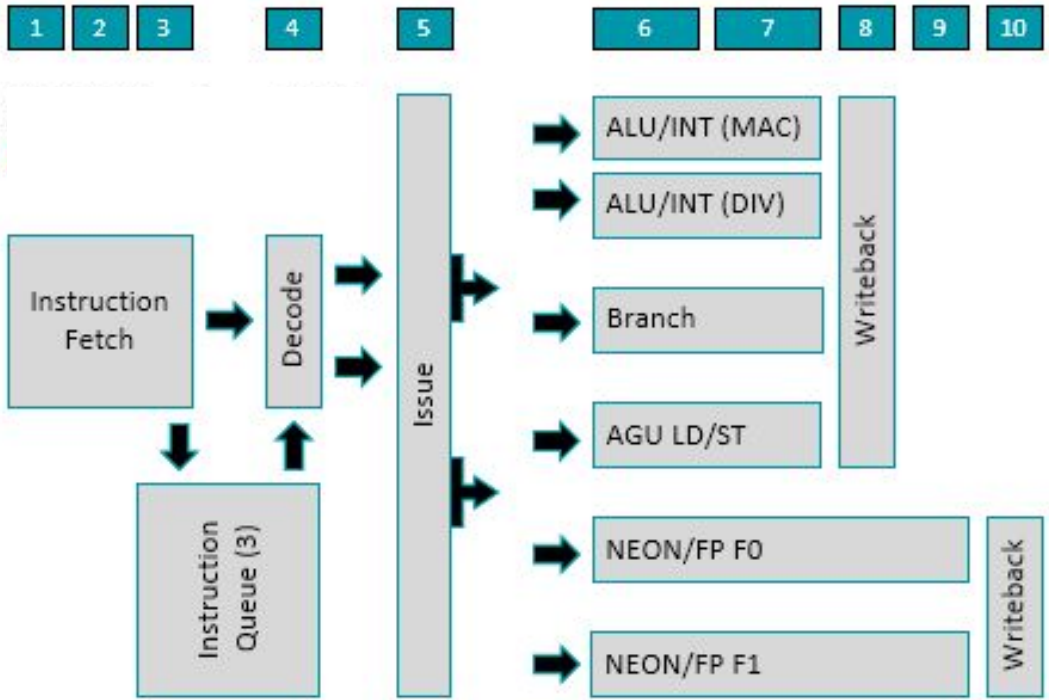
Dynamic Scheduling

Arquitectura de Computadoras
2025

ARM Cortex-A53 Microarchitecture



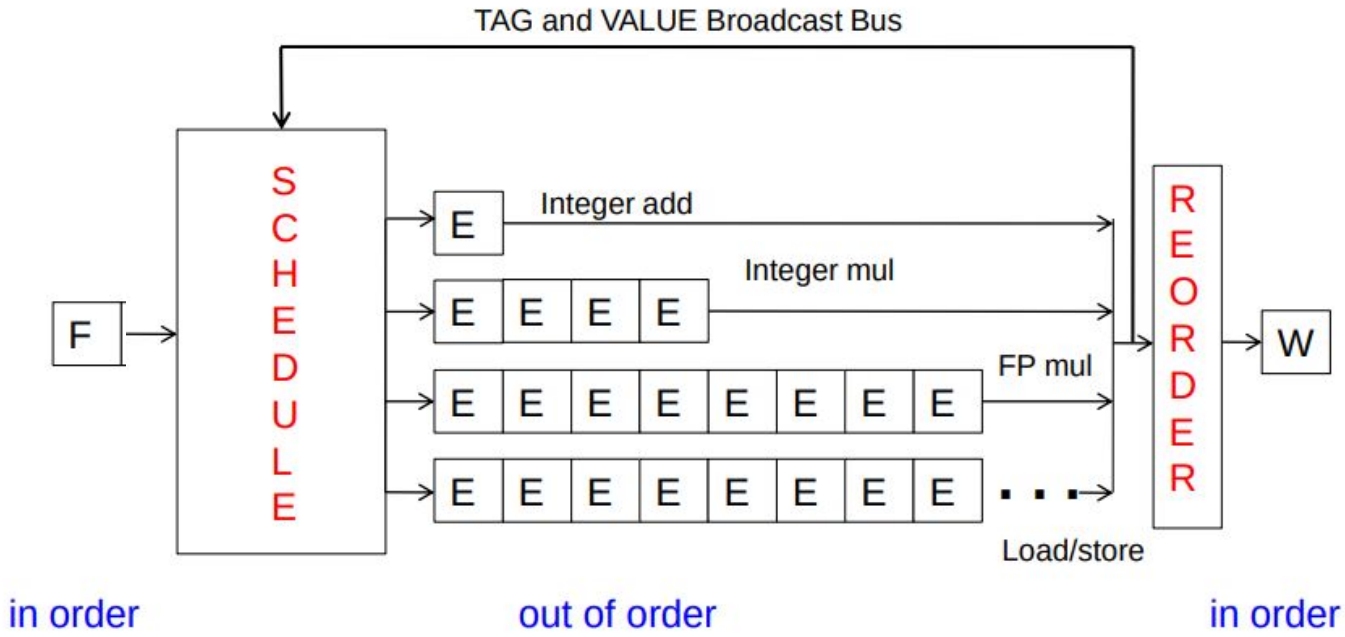
https://www.anandtech.com/show/11441/dynamiq-and-arms-new-cpus-cortex-a75-a55/4

Scheduling

Static Scheduling: se captura una instrucción (o un grupo de instrucciones) y se la ejecuta, a menos que haya una dependencia que no se pueda resolver, en ese caso, el pipeline se detiene (*stall*).

Dynamic Scheduling: Es una técnica donde el *hardware* reordena la ejecución de instrucciones para reducir los *stalls* manteniendo el flujo de datos y *exception behavior*.

Ejecución OoO (Out-of-Order)



Algoritmo de Tomasulo

Aunque existen muchas implementaciones, todas se basan en dos principios: **Determinación dinámica** de cuando una instrucción está lista para ejecutarse y **register renaming** para evitar dependencias de nombre.

Unidades Funcionales (FU)
Hardware que se encarga de realizar la operación propiamente dicha. Ej: Memoria, ALU, FP ALU, Multiplicadores, etc. Cada uno de ellos puede tener distintas latencias.

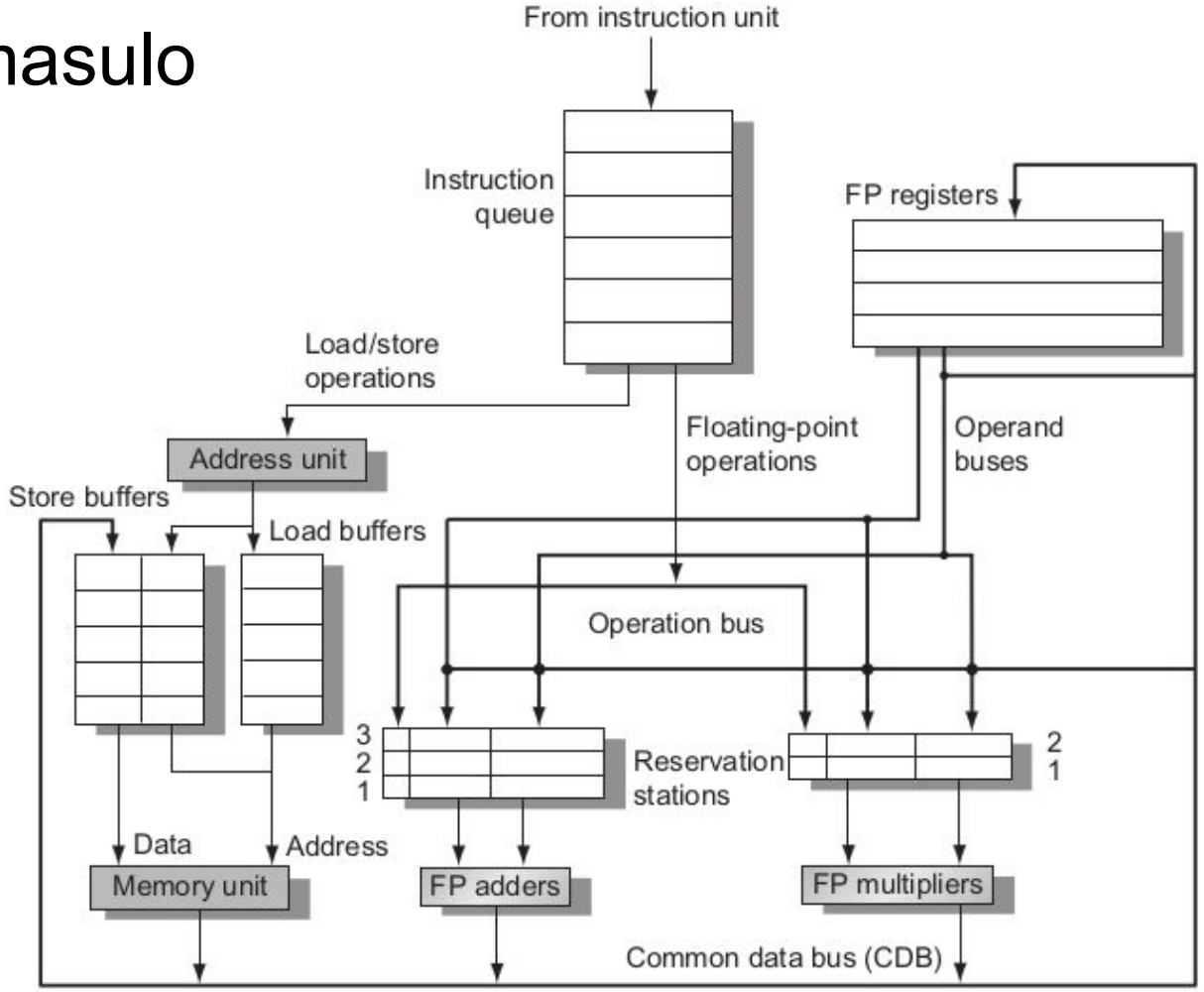
Reservation Stations (RS)
Registros distribuidos asociados a una unidad funcional que contiene la operación y los operandos necesarios.
La idea del algoritmo es capturar y “bufferear” los operandos tan pronto como estén disponibles con el objeto de no tener que recurrir a los registros.
En el caso en que existan instrucciones con dependencia de datos, se designa la reservation station que producirá el operando necesario

Algoritmo de Tomasulo

Estructura básica de una unidad de punto flotante RISC-V usando el algoritmo de Tomasulo

Register file

	Valid	Tag	Value
X0			
X1			
X2			
...			
x31			



Reservation Stations

Las reservation station tienen 7 campos:

- Op: Operación a realizarse entre los operandos
- Qj, Qk: Tag de la RS asociada a la UF que va a producir el operando necesario. Un valor de cero indica que el operando ya se encuentra disponible en Vj o Vk.
- Vj, Vk: Valores de los operandos. En las instrucciones load, Vk se utiliza para el offset.
- A: Contiene el resultado del cálculo de dirección para los load o store.

Name	Op	Qj	Vj	Qk	Vk	A	Busy

Register file tiene un campo Qi:

- Qi: Tag de la RS asociada a la UF que está calculando el resultado que debe almacenarse en este registro, si el valor es cero, implica que el resultado de ninguna instrucción activa, tiene como destino este registro. El valor es simplemente el contenido de este registro.

Algoritmo de Tomasulo

IF	IS/ID	Ex	Wb

Fetch
Se levantan las instrucciones de memoria y se colocan en una FIFO.

Issue y decode
- Se decodifican las instrucciones y si esta disponible la reservation station correspondiente, se le pasa la instrucción y los operandos (si están en registros).
- Si no hay una reservation station disponible, la instrucción debe esperar (stall) hasta que se libere.
- Si los operandos no están en los registros, se linkea con la reservation station que lo producirá y la instrucción queda en espera hasta que pueda ejecutarse.

Execute
- Cuando todos los operandos están listos, la operación puede ejecutarse en la unidad funcional correspondiente.
- Los loads y stores requieren, primero, que se calcule la dirección efectiva para luego acceder a la memoria.
- La dirección calculada se almacena en el buffer correspondiente en orden de ejecución del programa, esto previene hazard en la memoria.

Write Result
Cuando el resultado está disponible, se utiliza el CDB para escribirlo en los registros y en las reservation stations que lo necesitan.

Dependencias y hazard de datos

- **Real dependencia datos (hazard RAW)**:

```
add X0, X1, X2
add X3, X0, X4
```

```
add X0, X1, X2
stur X3, [X0, #0]
```

- **Dependencia de salida (hazard WAW)**:

```
add X0, X1, X2
add X0, X3, X4
```

```
add X0, X1, X2
ldur X0, [X3, #0]
```

- **Antidependencia de datos (hazard WAR)**:

```
add X0, X1, X2
add X1, X3, X4
```

```
add X0, X1, X2
ldur X1, [X3, #0]
```

Register renaming

```
1>          LDUR X0, [X20,#0]
2>          ADD X0, X0, X21
3>          STUR X0, [X20, #0]
```

- Se elige un registro no se está utilizando para almacenar datos necesarios (ej: x19)

- En la instrucción 2, cambiamos el registro destino para asegurarnos que el valor final del registro va a ser el correcto sin importar en qué orden se ejecuten las instrucciones.

- Se debe corregir el resto de las instrucciones para que el código no pierda sentido

```
1>          LDUR X0, [X20,#0]
2>          ADD X19, X0, X21
3>          STUR X19, [X20, #0]
```

Register renaming en el algoritmo de tomasulo se hace mediante las reservation stations:

En cuanto el operando está listo se lo escribe en la RS, si el mismo no está listo se escribe el nombre de la RS que producirá el resultado.

Dependencias

Dependencias de datos:

- Si existen operandos que no están listos, se designa la reservation station encargada de generar el operando. (Register Renaming).

- Dado que puede haber muchas más reservation stations qué registros reales, esta técnica puede eliminar hazards provenientes de dependencias de nombre que el compilador no puede detectar.

- Si varias escrituras se superponen en ejecución, solo la última se utiliza para actualizar el registro.

- Los resultados pasan directamente a la unidad funcional desde la reservation station donde esperan. Este bypass se realza mediante el bus de resultados en común (CDB)

Dependencia estructural:

- Si todos los operandos están listos pero no hay una FU libre, se debe esperar.

Dependencia de datos en out-of-order pipelines

Real dependencia de datos:
La instrucción Instr1 es dependiente en datos con la instrucción Instr2 cuando la instrucción Instr1 produce un resultado que debe ser utilizado por la instrucción Instr2.

Dependencias de nombre
Dos instrucciones usan el mismo registro o posición de memoria, pero no hay flujo real de datos.

- **Dependencia de salida:** Ocurre cuando las instrucciones Instr1 y Instr2 escriben la misma posición de memoria o registro. El orden original se debe preservar para asegurar que el valor final se corresponda con el valor de Instr2.

- **Antidependencia:** Cuando Instr2 escribe un registro o posición de memoria que la instrucción Instr1 necesita leer. El orden original se debe preservar para asegurar que *i* lea el dato correcto.

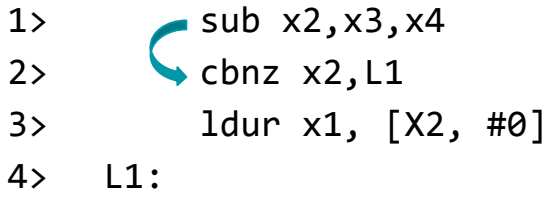
Dependencia de datos condicional

Una dependencia de datos condicional determina el ordenamiento de una instrucción respecto a otra de salto. De forma en que dicha instrucción se ejecuta en el correcto orden de programa y solo cuando debe ser.

Restricciones impuestas por la dependencia de control:

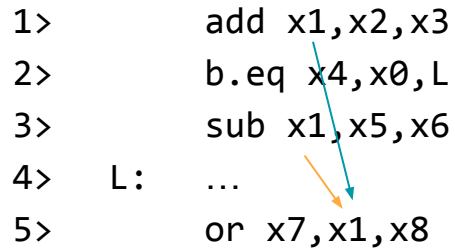
- Una instrucción que tiene una dependencia condicional sobre un salto no puede moverse antes que el salto de forma en que su ejecución no esté controlada por el salto.

- Una instrucción que no tiene una dependencia condicional sobre un salto no puede moverse después del salto de forma en que su ejecución esté controlada por el salto.



Dependencia de datos condicional

El orden del programa (condición del salto) es quien establece que predecesor realmente genera el dato necesario para la instrucción.



A partir de esta doble dependencia únicamente un hazard puede darse, aunque existan dos potenciales. Cual de ellos podría generarse, depende del resultado del salto condicional.

Resumen de hazard respecto a las arquitecturas

Tipo de dependencia	In-order (1-issue)	Multiple issue in-order	Multiple issue out-of-order
Estructurales (Hazard)	No*	No*	Puede provocar stalls
Datos de datos condicional	Puede provocar stalls	Deben considerarse al armar los issue packet. Puede provocar stalls	Deben considerarse al reordenar el código. Puede provocar stalls
Datos	RAW	RAW, WAW	RAW, WAW, WAR

*Pueden existir por criterios de diseño, debe especificarse el hardware y el problema en particular

RAW: Puede evitarse con stalls o forwarding
WAW: Puede evitarse con register renaming

Hazard Estructurales: En la ejecución out-of-order, puede ocurrir que una instrucción requiere una unidad funcional (ALU, memoria, etc) que está ocupado ejecutando otra instrucción.

Ejercicio dependencias en out-of-order pipelines

```
1> Loop:    LDUR X0, [X20,#0]
2>          ADD X0, X0, X21
3>          STUR X0, [X20, #0]
4>          SUBI X20, X20, #8
5>          CMP X20, X22
6>          B.GT Loop
```

Instrucciones involucradas	Registro en conflicto	Tipo de dependencia
1, 2	X0	Datos (RAW)
1, 2	X0	Datos (WAW)
2, 3	X0	Datos (RAW)
2, 1	X0	Datos (WAW - condicional)
1, 4	X20	Datos (WAR)
3, 4	X20	Datos (WAR)
4, 5	X20	Datos (RAW)
4, 1	X20	Datos (RAW - condicional)
4, 3	X20	Datos (RAW - condicional)
1,2,3,4,5 con 6	-	Control

Ejercicio 1

Mostrar el contenido de las tablas de *Status,* las *Reservation stations* y el flujo de ejecución para la siguiente secuencia de código cuando se haya completado y escrito el resultado del primer load.
Considerando:

- Issue = 1 instrucción
- Load = 6 *reservation stations* / 1 clock cycle
- Store = 6 *reservation stations* / 1 clock cycle
- Suma punto flotante = 3 *reservation stations* / 2 clock cycles
- Multiplicación punto flotante = 2 *reservation stations* / 6 clock cycles

```
1> LDURD D6, [X2, #32]
2> LDURD D2, [X3, #44]
3> FMULD D0, D2, D4
4> FSUBD D8, D2, D6
5> FDIVD D0, D0, D6
6> FADDD D6, D8, D2
```

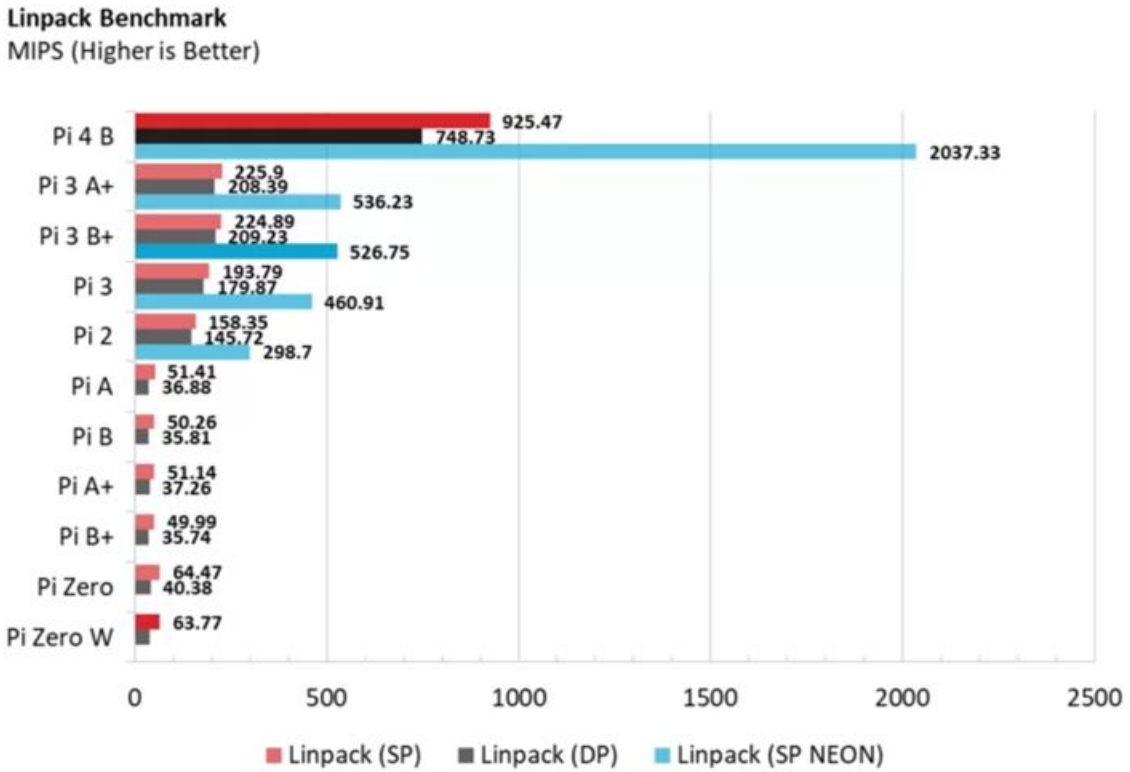
Ejercicio 2

Para la siguiente secuencia de código, asumiendo que se han emitido todas las instrucciones de dos iteraciones sucesivas del bucle, pero ninguna de las operaciones load/store de punto flotante se ha completado, mostrar el contenido de las tablas de *Status,* las *Reservation stations* y el flujo de ejecución. Considerando:

- El salto fue predicho correctamente como *taken.*
- Issue = 2 instrucciones
- Load = 6 *reservation stations* / 1 clock cycle
- Store = 6 *reservation stations* / 1 clock cycle
- Suma entera = 3 *reservation stations* / 1 clock cycle
- Suma punto flotante = 3 *reservation stations* / 2 clock cycles
- Multiplicación punto flotante = 2 *reservation stations* / 3 clock cycles

```
1> Loop:    LDURD D0, [X1, #0]
2>          FMULD D4, D0, D2
3>          STURD D4, [X1, #0]
4>          SUBI  X1, X1, #8
5>          CBNZ  X1, Loop
```

Linpack Benchmark



https://www.tomshardware.com/reviews/raspberry-pi-4-b,6193.html

Hardware basado en especulación

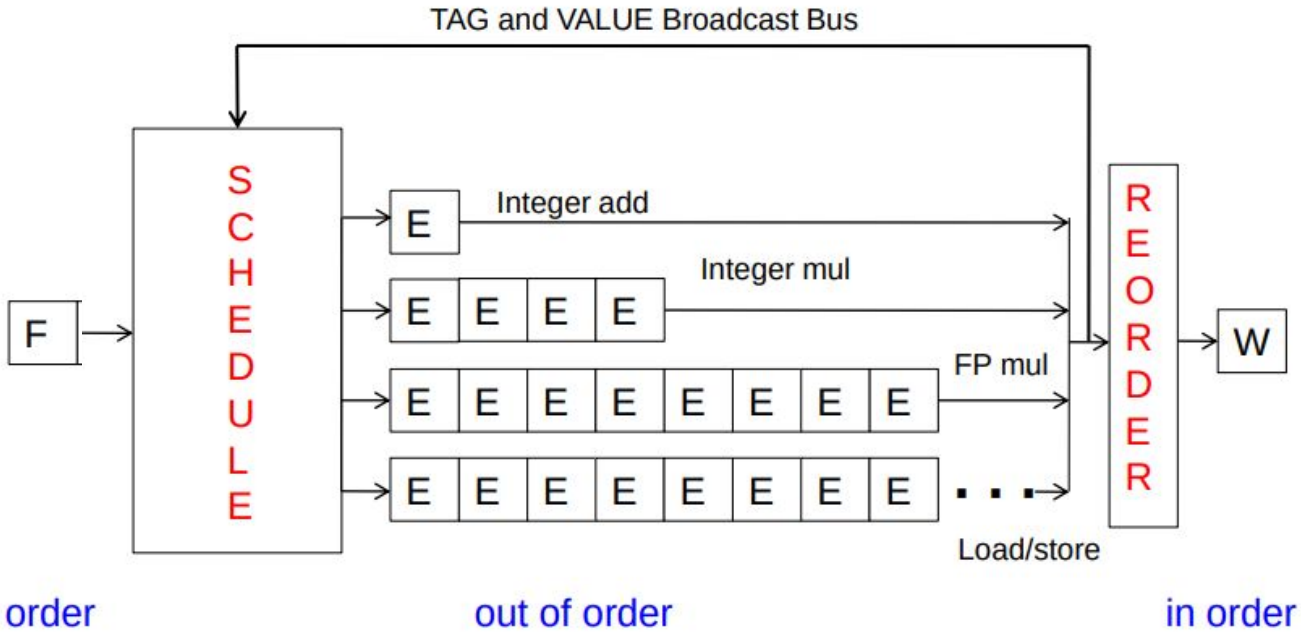
Para superar las dependencias de control se especula el resultado de un branch condicional y se continúa ejecutando el programa como si la predicción fuera correcta

Tomasulo utilizaba la predicción para hacer fetch y decode de las instrucciones siguientes a un branch.

En especulación se hace fetch, decode, issue y execute a partir de la predicción de saltos. Es necesario agregar hardware para solucionar las predicciones erróneas

ReOrder Buffer (ROB)

Es necesario agregar un ROB para asegurar que las excepciones se levanten correctamente.

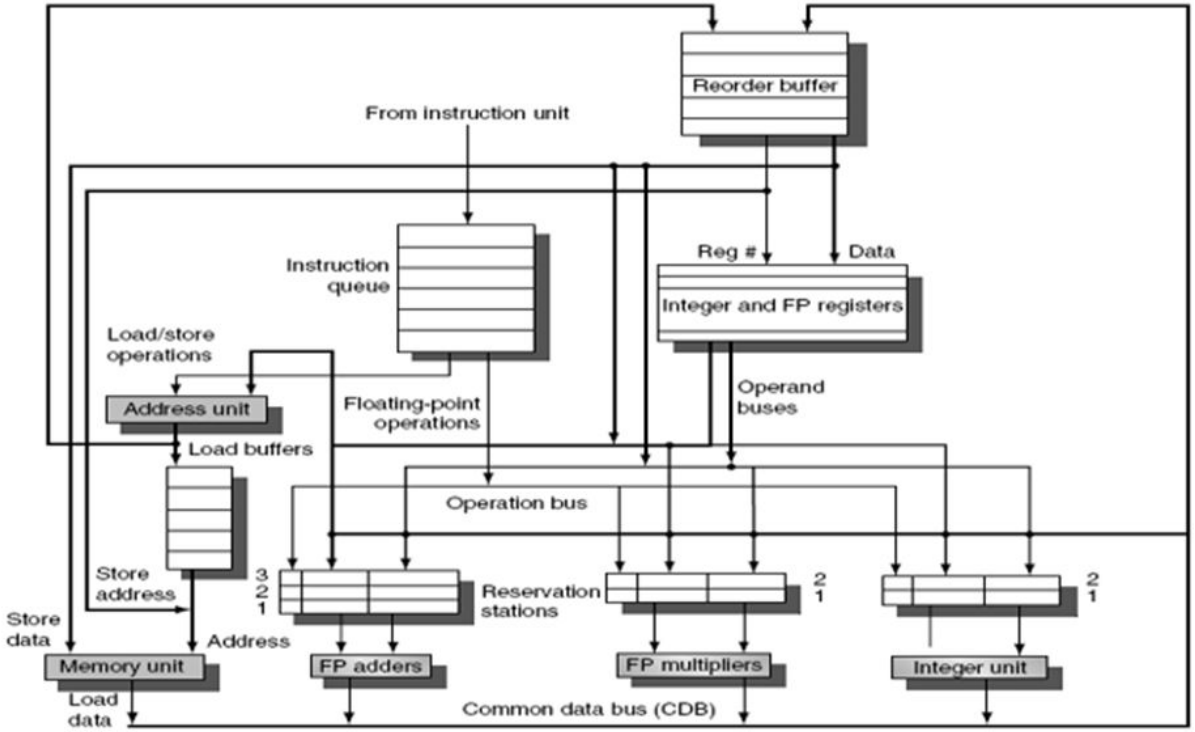


Tomasulo + Speculation

Se agrega un buffer de reordenamiento (ROB) que mantiene los resultados hasta que el salto se termina de ejecutar.

Se agrega una fase de la instrucción: *commit* en la que se actualiza la memoria y los registros cuando una instrucción no es más especulativa.

Cada línea del ROB contiene: Tipo de instrucción, campo de destino, campo de valor y un campo de ready.



Intel x86 Sandy Bridge micro-architecture

